Infraestrutura de Hardware

Instruções lógicas e de desvio no MIPS

Prof. Adriano Sarmento

Centro de Informática UFPE

Universidade Federal de Pernambuco

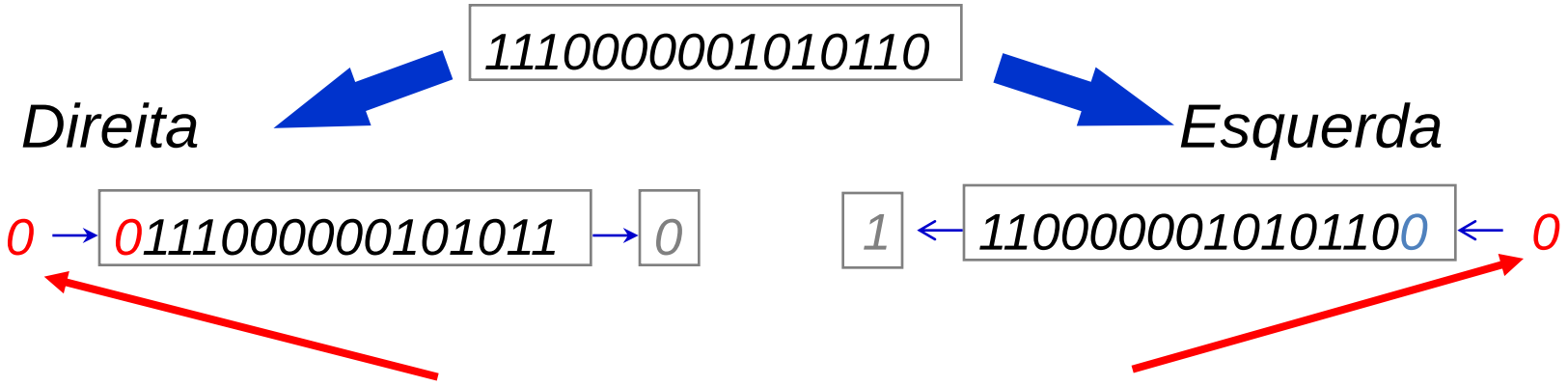
Operações Lógicas

- Permitem manipulação bit a bit dos dados
- Úteis para extrair ou inserir um grupo de bits em uma palavra
  - Podem modificar o formato de um dado

Operação	C	Java	MIPS
Logical Shift left	<<	<<	`sll`
Logical Shift right	>>	>>>	`srl`
Bitwise AND	&	&	`and, andi`
Bitwise OR	|	|	`or, ori`
Bitwise NOT	~	~	`a nor 0`

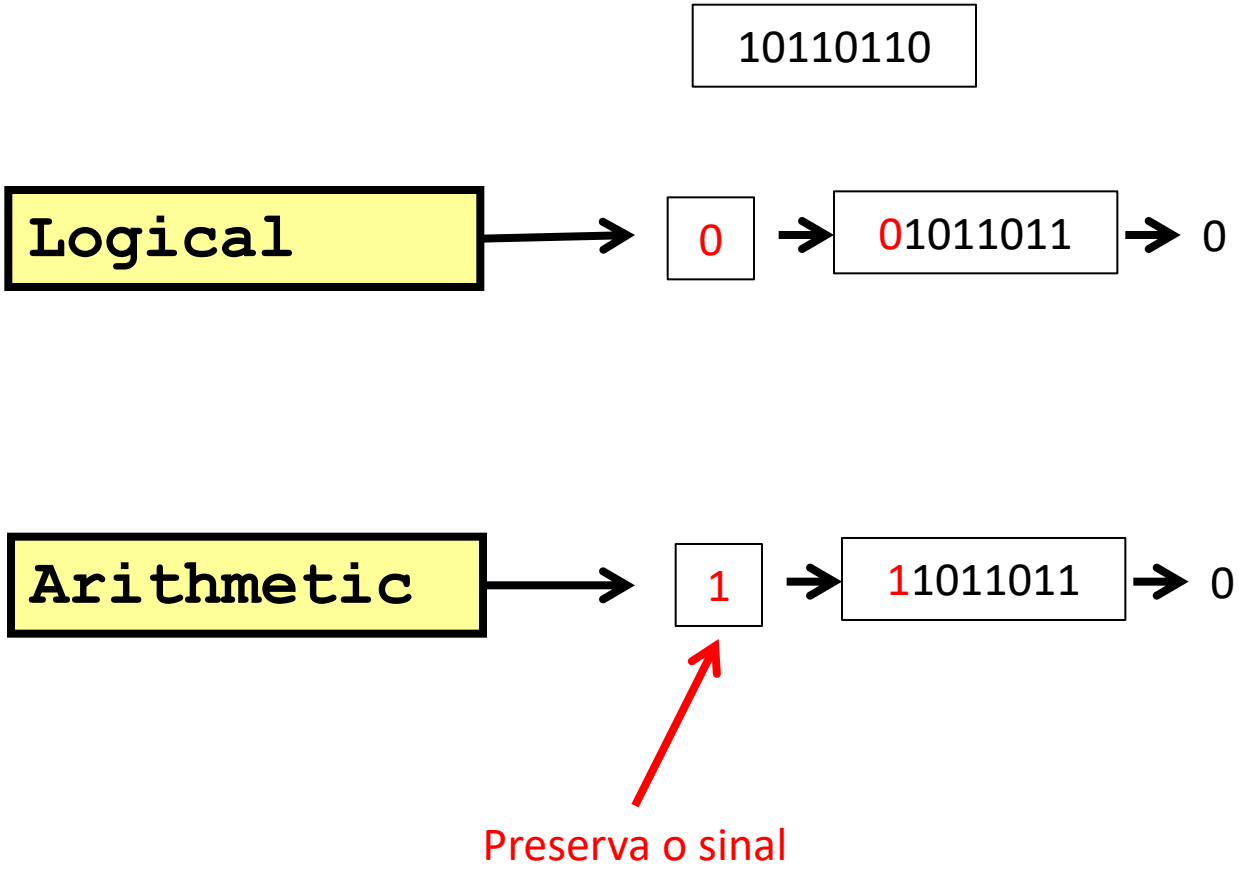
Operações Lógicas de Deslocamento (Shift)

- Afeta a localização dos bits em um dado
- Permite o deslocamento para esquerda ou direita de bits de um dado
- Insere grupo de bits no dado



Preenche com 0's a quantidade de bits deslocados

Logical Shift Right x Arithmetic Shift Right



Operações Lógicas de Deslocamento no MIPS

- Instruções de deslocamento no MIPS possuem 3 operandos:
  - destino, fonte, quantidade de bits deslocados

```
sll a,b,3
```

→

```
a = b << 3
```

```
srl a,b,4
```

→

```
a = b >> 4
```

- Deslocamento para esquerda (direita) de i bits de um valor é equivalente a multiplicar (dividir) valor por 2^i

Multiplicação com `SLL`

- Multiplicação do valor por 4 (2^2)

Código C

```
g = g * 4;
```

g em `$s1`

Código Assembly MIPS

```
sll $s1, $s1, 2
```

g armazena valor 4 (100_2)

0000000000000100

g armazena valor 16 (10000_2)

0000000000010000

Formato da Instrução `SLL` e `SRL`

Formato R de Instrução

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

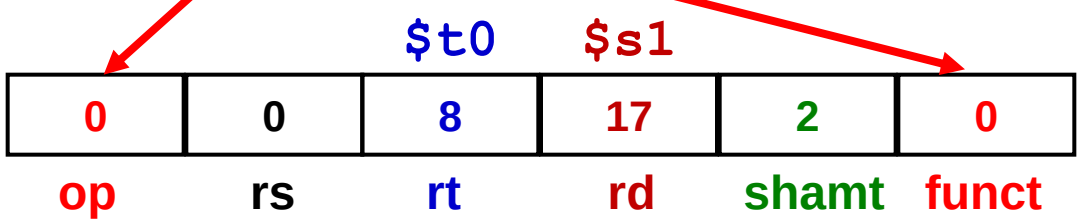
op	opcode da instrução
rs	não utilizado para **`sll`** e **`srl`**
rt	registrador que contém operando fonte
rd	registrador destino que contém resultado
shamt	shift amount, quantidade de bits deslocados
funct	função (function) que estende o opcode

Representando SLL na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
sll $s1,$t0,2
```



Outras Operações Lógicas

- AND ,OR, NOT (MIPS implementa como *A NOR 0*)
- Úteis para extrair grupos de bits
  - “Máscara” para encontrar padrões de disposição de bits
- No MIPS, possui 3 operandos como (ADD) e tem formato R

```
and a,b,c
```

→

```
a = b & c
```

```
and $t0,$t1,$t2
```

$t2	0000 0000 0000 0000 0000 1101 1100 0000
$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	0000 0000 0000 0000 0000 1100 0000 0000

Operações de Controle de Fluxo

- Alterar a sequência de execução das instruções:

Ling. alto nível

- ***If ...then ...else***
- ***case***
- ***loop***
- ***go to***

Linguagem máquina

- ***Desvio condicional a comparações entre variáveis e/ou valores***
- ***Desvio incondicional***

Desvios no MIPS

- Consegue-se através de dois tipos de instruções
  - ***Branch*** (desvio condicional)
  - ***Jump*** (desvio incondicional)
- Instruções do tipo *branch* tem 3 operandos
  - fonte 1, fonte 2, label de instrução

```
beq a,b,L1
```

se (a ==b) desvie para L1

```
bne a,b,L2
```

se (a !=b) desvie para L2

- Instruções do tipo *jump* tem um único operando, o label

```
j L3
```

desvie para L3

Implementando Desvio Condicional - `if`

Código C

```
if (i == h)
  i  = g + h;
else
  i = g - h;
```

`g` em `$s1`, `h` em `$s2`, `i` em `$s3`

Assembler calcula endereço

Código Assembly MIPS

```
bne $s3, $s2, Else
add $s3, $s1, $s2
j Exit
Else: sub $s3,$s1,$s2
Exit:...
```

Implementando Loops - `while`

Código C

```
while (save[i] == h)
  i += 1;
```

`h` em `$s2`, `i` em `$s3` e endereço base de `save` em `$s4`

Código Assembly MIPS

```
Loop: sll $t1,$s3,2
add $t1, $t1, $s4
lw $t0,0($t1)
bne $t0,$s2,Exit
addi $s3, $s3,1
j Loop
Exit:...
```

← `$t1= i*4`

← `End_base + $t1`

Mais Sobre Branchs no MIPS

- O operando relativo ao label nas instruções de branch corresponde na verdade ao deslocamento em relação ao endereço da instrução contida no PC (Program Counter)
  - **PC já incrementado de 4!**
  - **PC = PC + (deslocamento * 4) se reg1 == reg2**

```
beq rs,rt,deslocamento
```

Formato I de Instrução

op	rs	rt	deslocamento
6 bits	5 bits	5 bits	16 bits

Representando BNE na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

```
Loop: sll $t1,$s3,2
add $t1, $t1, $s4
lw $t0,0($t1)
bne $t0,$s2,Exit
addi $s3, $s3,1
j Loop
Exit:...
```

	$t0	$s2	Exit
5	8	18	2
op	rs	rt	deslocamento

Mais Sobre Jumps no MIPS

- O operando relativo ao label nas instruções de jumps corresponde na verdade ao endereço (número) da instrução a ser executada
  - **PC = endereço**

```
j endereco
```

Formato J de Instrução

op	endereço
6 bits	26 bits

Mais Operações Condicionais no MIPS

- Armazene 1 se condição é verdade, senão 0
  - **Set less than**

```
slt rd,rs,rt
```

se (rs < rt) rd = 1, senão rd = 0

- MIPS possui registrador que armazena valor 0
  - **`$zero`**

- **`slt`** pode ser utilizada junto com **`beq, bne`**

```
slt $t0,$s1,$s2
bne $t0,$zero, L1
```

se ($s1 < $s2) desvie para L1

Mais Operações Condicionais no MIPS

- Nenhuma instrução de desvio para <, >=, … ?????
  - Combinar em uma só instrução branch e comparações
  - (>, <, >= …) , requer mais trabalho por instrução
  - Clock mais lento
  - Penaliza todas as instruções
  - Branch se == ou != é o mais comum